

MoneyPAL NEST Genesis Layer Development Brief

Legacy System Onboarding and Canonical Migration Framework

Version 1.0

Purpose

The Genesis Layer is the onboarding framework for the MoneyPAL NEST platform.

Its purpose is to transform an existing financial institution's operational data from legacy systems into the MoneyPAL Canonical Domain Model (CDM) and initialize the NEST Aggregate Block (NAB) architecture.

The Genesis Layer performs a one-time migration during the implementation of a Marketplace Provider or Lender onto MoneyPAL. Once migration is complete, the Genesis Layer is no longer involved in day-to-day business processing.

The Genesis Layer ensures that institutions can preserve their business history while transitioning from traditional relational systems to an event-driven, append-only distributed architecture.

Objectives

The Genesis Layer shall:

- Connect to multiple legacy source systems.
- Extract business data without modifying source systems.
- Transform relational records into canonical business aggregates.
- Generate Genesis Events for every aggregate.
- Initialize NEST Aggregate Blocks (NABs).
- Build the first generation of analytics projections.
- Populate the Regulatory Repository.
- Populate the Economic Knowledge Graph.
- Populate the AI Feature Store.
- Produce reconciliation and audit reports.

- Support repeatable migrations in development and test environments.
-

Design Principles

The Genesis Layer shall:

- Be read-only with respect to source systems.
 - Be configuration-driven rather than hardcoded.
 - Support multiple source systems.
 - Support parallel execution.
 - Produce complete audit trails.
 - Preserve financial integrity.
 - Be restartable from checkpoints.
 - Validate every transformation.
 - Be reusable for future marketplace onboarding.
-

Supported Source Systems

The Genesis Layer shall initially support:

Core Lending Systems

- Oracle-based Loan Management Systems
- Oracle Core Banking Systems
- Microsoft SQL Server-based lending systems
- PostgreSQL-based lending platforms

Accounting Systems

- Tally ERP
- TallyPrime

Microfinance and NBFC Platforms

- Prosper (LCode Technologies)
- Custom Loan Origination Systems
- Custom Loan Management Systems

Flat File Sources

- CSV
- Excel
- Fixed-width files
- XML
- JSON

Additional connectors shall be implemented through a common adapter framework.

High-Level Architecture

Legacy Systems

Oracle
Prosper
Tally
CSV
Excel



Connector Framework



Extraction Engine



Canonical Mapping Engine



Validation Engine



Genesis Event Generator



NAB Initializer

|

Projection Builder

|

Analytics Repository
Regulatory Repository
Knowledge Graph
AI Feature Store

Connector Framework

Every connector shall implement a standard interface.

Functions:

- Connect
- Authenticate
- Discover schema
- Extract master data
- Extract transactional data
- Resume extraction
- Validate extraction
- Generate extraction statistics

The framework should allow new connectors to be added without modifying migration logic.

Canonical Mapping Engine

The Mapping Engine converts legacy entities into Canonical Domain Model aggregates.

Example mappings:

Legacy Object	Canonical Aggregate
Customer	Person NAB
Business Customer	Enterprise NAB
Joint Account Holder	Relationship NAB

Loan Account	Loan NAB
Guarantor	Relationship NAB
Collateral	Asset NAB
Collection Record	Workflow NAB
Branch	Marketplace Organization
Employee	Person NAB
Ledger Account	Financial Account Aggregate

Mapping definitions shall be declarative and externally configurable.

Tally Mapping

The Tally connector should extract:

- Chart of Accounts
- Customer Ledger
- Vendor Ledger
- General Ledger
- Sales
- Purchases
- GST Information
- Journal Entries
- Receivables
- Payables
- Cash Book
- Bank Book

Derived canonical objects include:

- Enterprise Financial Profile
- Cash Flow History
- Income Statement
- Working Capital Indicators
- Business Health Metrics

These become inputs for analytics and AI.

Prosper Mapping

The Prosper connector should extract:

- Borrowers
- Groups
- Centres
- Loans
- Repayments
- Collections
- Staff
- Branches
- Products
- Delinquency
- Risk Classification
- Portfolio Statistics

Derived canonical objects include:

- Enterprise NAB
 - Household NAB
 - Person NAB
 - Loan NAB
 - Workflow NAB
 - Relationship NAB
-

Validation Framework

Validation stages include:

Structural Validation

- Mandatory fields
- Data types
- Key uniqueness
- Referential integrity

Business Validation

- Loan balances
- Outstanding principal
- Interest calculations
- Repayment schedules

- Product mapping

Financial Validation

- Portfolio totals
- Trial balance consistency
- Ledger balancing
- Branch balancing

Canonical Validation

- Aggregate completeness
- Relationship integrity
- Public Key generation
- Event sequencing

No record should be migrated without validation status.

Genesis Event Generation

Legacy systems generally represent current state rather than event history.

The Genesis Layer synthesizes a Genesis Event for every aggregate.

Examples:

Enterprise Created

Loan Imported

Household Initialized

Asset Registered

Consent Initialized

Marketplace Registered

Relationship Established

These events become Block 1 of every NEST Aggregate Block.

NAB Initialization

For every aggregate:

Generate Public Key

↓

Create Aggregate

↓

Generate Genesis Event

↓

Create Block 1

↓

Generate Current Projection

↓

Publish Event

The Genesis Layer does not recreate historical block-by-block evolution unless reliable historical events are available.

Historical Data Strategy

Where historical transactions are available, they should be migrated as ordered historical events.

Examples:

Loan Disbursements

Repayments

Collections

Restructuring

Interest Accrual

Write-offs

If historical granularity is unavailable, create summarized migration events while preserving accounting accuracy.

Projection Initialization

Following migration, automatically initialize:

- Regulatory Repository
- Marketplace Analytics Repository
- Operational Dashboards
- Economic Knowledge Graph
- AI Feature Store
- Portfolio Benchmarks

No manual intervention should be required.

Reconciliation Engine

The Genesis Layer shall reconcile:

- Number of borrowers
- Number of enterprises
- Number of households
- Loan count
- Outstanding principal
- Outstanding interest
- Collections
- Product balances
- Branch balances
- Portfolio totals
- General Ledger balances (where applicable)

Any discrepancy must produce an exception report.

Migration Audit Repository

Each migration execution records:

- Migration ID
- Institution
- Marketplace
- Source System
- Source Version
- Connector Version
- Mapping Version
- Operator
- Start Time
- End Time
- Records Read
- Records Migrated
- Records Rejected
- Validation Results
- Reconciliation Results
- Checksums

This repository provides complete migration traceability.

Restart and Recovery

Support:

- Checkpointing
 - Resume after interruption
 - Partial reruns
 - Rollback of incomplete migrations
 - Dry-run execution
 - Incremental testing
-

Security

The Genesis Layer shall:

- Operate with least privilege.
- Encrypt extracted data.
- Mask PII in test environments.
- Audit all access.
- Never expose credentials.
- Delete temporary migration files after successful completion.
- Support secure credential vault integration.

APIs

Expose administrative APIs for:

- Start migration
- Stop migration
- Resume migration
- Migration status
- Validation reports
- Reconciliation reports
- Audit reports
- Projection rebuild

These APIs are administrative and not exposed to end users.

Deliverables

The Genesis Layer development team shall deliver:

1. Connector Framework
 2. Oracle Connector
 3. Tally Connector
 4. Prosper Connector
 5. CSV/Excel Connector
 6. Canonical Mapping Engine
 7. Validation Framework
 8. Genesis Event Generator
 9. NAB Initializer
 10. Projection Initializer
 11. Reconciliation Engine
 12. Migration Audit Repository
 13. Monitoring Dashboard
 14. Administrative APIs
 15. Configuration Toolkit for custom mappings
 16. Developer SDK for additional connectors
-

Definition of Success

The Genesis Layer implementation is considered successful when:

- A legacy institution can be onboarded without modifying its source systems.
- All supported source systems are transformed into Canonical Domain Model aggregates.
- Every aggregate is initialized with a valid Genesis Event and NEST Aggregate Block.
- Financial balances reconcile exactly with the legacy system.
- The Regulatory Repository, Marketplace Analytics Repository, Economic Knowledge Graph and AI Feature Store are generated automatically from migrated aggregates.
- Migration execution is fully auditable, repeatable and restartable.
- The institution can transition to native NEST operations immediately after migration without dependence on the legacy platform.
- All subsequent business activity is captured exclusively through native NEST Aggregate Block events.
- The projected Regulatory Repository contains sufficient, reconciled and regulator-ready information to generate **DNBS-4A** and **DNBS-4B** returns (or their successor regulatory formats) without requiring access to the legacy Oracle, Prosper or Tally systems.
- A regulator, auditor or implementation partner can independently verify that the migrated portfolio, regulatory returns and financial statements are consistent with the legacy system through documented reconciliation reports and immutable Genesis audit records.

The Genesis Layer is therefore the permanent institutional onboarding capability of the Moneypal NEST platform, enabling any lender, NBFC, cooperative financial institution or marketplace provider to transition from legacy operational systems into the distributed, event-driven NEST ecosystem while preserving historical integrity, regulatory continuity and future interoperability.

Moneypal Regulatory Information Model (RIM)

Chapters 11–15: Regulatory Projection Specifications

Version 1.0 (Developer Framework)

Chapter 11 — DNBS-02 Projection Specification

Purpose

The DNBS-02 Projection generates the statutory return describing the financial position and loan portfolio of the NBFC.

This projection is derived entirely from the Regulatory Information Model (RIM), which in turn is projected from:

- NEST Aggregate Blocks (NABs)
- Financial Account Aggregates (FAAs)
- Journal Aggregates
- Regulatory Case Aggregates (RCAs)

The DNBS-02 generator shall never query operational databases directly.

Primary Data Sources

Enterprise NAB

Loan NAB

Financial Account Aggregate

Journal Aggregate

Regulatory Case Aggregate

Product Aggregate

Marketplace Aggregate

Projection Pipeline

NAB Events

↓

Financial Postings

↓

Regulatory Information Model

↓

Validation



DNBS-02 Projection



Submission Package

Major Reporting Sections

The projection engine shall populate:

- Institution Profile
 - Capital Funds
 - Borrowings
 - Loan Portfolio
 - Investments
 - Asset Classification
 - Income
 - Expenditure
 - Profitability
 - Exposure
 - Provisioning
 - Off-Balance Sheet Items
-

Core Metrics

Examples include:

- Total Assets
- Total Loan Portfolio
- Gross Outstanding
- Net Outstanding
- Gross NPA
- Net NPA
- Provision Held
- Capital Funds
- Borrowings
- Investments

Every metric shall be traceable to the originating NAB events.

Chapter 12 — DNBS-04A Projection Specification

Purpose

Generate the Dynamic Liquidity Statement required for regulatory supervision.

The report measures expected inflows and outflows across prescribed maturity buckets.

Source Aggregates

Loan NAB

Financial Account Aggregate

Repayment Schedule Aggregate

Investment Aggregate

Borrowing Aggregate

Journal Aggregate

Regulatory Dimensions

Counterparty

Product

Liquidity Category

Maturity Bucket

Interest Type

Funding Source

Marketplace

Liquidity Buckets

Projection engine shall support configurable buckets.

Example:

- Overnight
- 2–7 Days
- 8–14 Days
- 15–30 Days
- 1–2 Months
- 2–3 Months
- 3–6 Months
- 6–12 Months
- 1–3 Years
- More than 3 Years

The bucket definitions shall be configurable.

Projection Logic

For every financial instrument:

Determine contractual maturity

↓

Determine expected cash flow

↓

Apply behavioural adjustments

↓

Assign maturity bucket

↓

Aggregate

↓

Validate

↓

Publish

Output Objects

Cash Inflows

Cash Outflows

Net Liquidity Position

Cumulative Gap

Liquidity Ratio

Stress Indicators

Regulatory Case Generation

Whenever:

- Liquidity Gap exceeds threshold
- Negative cumulative gap occurs
- Funding concentration exceeds limits

Generate a Regulatory Case Aggregate.

Chapter 13 — DNBS-04B Projection Specification

Purpose

Generate the Structural Liquidity and Interest Rate Sensitivity projections.

The objective is to assess long-term liquidity risk and exposure to changes in interest rates.

Primary Inputs

Loan NAB

Financial Account Aggregate

Journal Aggregate

Borrowing Aggregate

Investment Aggregate

Interest Rate Reference Data

Projection Dimensions

Interest Type

Fixed/Floating

Residual Maturity

Repricing Date

Counterparty

Product

Currency

Projection Process

Identify all interest-bearing assets.

↓

Identify all interest-bearing liabilities.

↓

Calculate repricing profile.

↓

Aggregate into regulatory buckets.

↓

Compute gaps.

↓

Generate sensitivity measures.



Generate Regulatory Cases if thresholds are exceeded.

Derived Measures

Interest Sensitive Assets

Interest Sensitive Liabilities

Gap

Cumulative Gap

Gap Ratio

Rate Shock Impact

Economic Value Impact

Regulatory Cases

Examples:

Interest Rate Mismatch

Funding Concentration

Long-term Liquidity Deficit

Excess Short-term Funding

These become independent Regulatory Case Aggregates.

Chapter 14 — DNBS-10 Projection Specification

Purpose

Generate information required for statutory auditor certification and regulatory compliance reporting.

Data Sources

Regulatory Information Model

Financial Account Aggregates

Journal Aggregates

Regulatory Case Aggregates

Audit Events

Consent Events

Projection Scope

Financial Position

Provisioning

Asset Classification

Policy Compliance

Accounting Controls

Regulatory Exceptions

Audit Trail

Internal Controls

Validation Framework

The projection engine shall verify:

Financial consistency

Journal balancing

Asset classification

Provision adequacy

Interest recognition

NPA treatment

Policy compliance

Any validation failure creates a Regulatory Case Aggregate.

Audit Lineage

Every reported value shall be traceable through:

Report Cell

↓

RIM Object

↓

Financial Account

↓

Journal

↓

Business Event

↓

NAB Block

Chapter 15 — DNBS-13 Projection Specification

Purpose

Generate regulatory disclosures relating to overseas investments and cross-border exposures where applicable.

For institutions with no overseas exposure, the projection engine shall produce a compliant nil return.

Source Aggregates

Investment Aggregate

Organization Aggregate

Financial Account Aggregate

Journal Aggregate

Regulatory Case Aggregate

Projection Dimensions

Country

Currency

Investment Type

Counterparty

Exposure Type

Accounting Classification

Regulatory Classification

Projection Logic

Identify overseas exposures.

↓

Classify by regulatory category.

↓

Aggregate.



Validate.



Generate report.

Validation Rules

Country codes valid

Currency codes valid

Exposure reconciles to Financial Accounts

Exposure reconciles to Investments

No orphan investments

Common Regulatory Projection Framework

All regulatory reports shall use the same processing lifecycle.

NAB Events



Financial Account Aggregates



Journal Aggregates



Regulatory Information Model



Projection Engine



Validation Engine



Regulatory Case Generation



Report Formatter



Submission Package

No report shall contain business logic.

All calculations shall be implemented within the Projection Engine and Rule Engine.

Common Traceability Model

Every value appearing in any regulatory return shall support complete lineage.

Report Cell



Regulatory Fact



Projection Rule



Financial Account



Journal Entry

↓

Business Event

↓

NEST Aggregate Block

This lineage shall be available through developer APIs and administrative tools for audit, reconciliation and inspection.

Common Developer Requirements

All projection modules shall:

- Be configuration-driven.
 - Support report versioning.
 - Separate business rules from presentation.
 - Produce machine-readable and human-readable outputs.
 - Support replay from historical NAB events.
 - Generate detailed reconciliation reports.
 - Log every execution with report version, rule version and source event range.
 - Allow new RBI reporting formats to be introduced without changes to the underlying NAB or Financial Account models.
-

Framework Success Criteria

The Regulatory Projection Framework is successful when:

1. Every regulatory return is generated exclusively from the Regulatory Information Model.
2. No report queries operational business databases directly.
3. Every reported value is fully traceable to immutable NAB events.
4. Regulatory Cases are automatically created for breaches, validation failures and supervisory exceptions.
5. Report definitions are configuration-driven and version-controlled.
6. New regulatory templates can be implemented by defining projection metadata and rules rather than rewriting application code.
7. Financial totals reconcile with Financial Account Aggregates and Journal Aggregates.

8. The framework supports the generation of applicable supervisory returns for NBFCs under ₹500 crore in assets, including DNBS-02, DNBS-04A, DNBS-04B, DNBS-10 and DNBS-13, with the ability to evolve as RBI reporting requirements change.